# Speeding Up Integer Programs with FPUs

Krishnan R: CS24BTECH11036
Maka Nehith: CS24BTECH11040

## 1 Abstract

The aim of this project is to incorporate FPU operations into programs that operate only on integers, to decrease their overall execution time. This can be achieved by utilizing the idle FPU to perform some operations in parallel with the integer unit.

## 2 Introduction

In many CPUs, when FPUs are not being used by the program, they can be exploited to perform some operations in parallel. That is, the idle FPU can be utilized in those programs to perform some operations in parallel on these integers by converting them to float and then converting the result back to integers after the operation is performed by the FPU. This could speedup the execution time of the program. This project demonstrates the above process by showing an improvement in execution time of some programs that operate exclusively on integers.

## 3 Hardware Setup

Modifications of the original C code to incorporate floating point (FP) operations depends on the underlying processor architecture, particularly the number of floating-point units (FPUs), number of integer units, and the latencies of their instructions. Modifications must be made taking into account the processor's specifications to achieve the maximum improvement in overall execution time.

### 3.1 Processor Configuration

RISC Processor is used for the execution of programs in this project. Modern CISC processors have deeply pipelined, superscalar integer execution units with out-of-order scheduling, making the integer operations highly optimized, so it is difficult to achieve an improvement in execution time. In addition, CISC architecture includes many operation-specific instructions with varying latencies, making it difficult to understand the assembly code and modify the original code using FP operations. However, RISC processors rely on simpler instructions and greater use of pipelining, which allows for potential performance gains through instruction-level parallelism and efficient use of floating-point units.

In this project, the GEM5 simulator is used to execute the programs on the RISC-V processor. This tool simulates the RISC-V processor. Moreover, RISC-V **Out-Of-Order execution(O3)** CPU is used, which allows instruction-level parallelism, thereby providing chances for improvement in performance upon using FP operations. Upon executing a program in the GEM5 simulator, a stats.txt file is generated, which provides various performance statistics, including the execution time (simulation time). This simulation time is used to analyze and compare the execution time of different programs. The details of execution stage functional units(multi-issue pipelines) in the RISC-V Out-Of-Order processor which is simulated on GEM5 are specified below.

| **Unit** | **Count** | **Operations (Latency)** |
|---|---|---|
| **Integer ALU (IntALU)** | 6 | Integer ALU Operations<br>(add, sub, etc.) – 1 cycle |
| **Integer Multiply/Divide (IntMultDiv)** | 2 | IntMult – 3 cycles<br>IntDiv – 20 cycles (non-pipelined) |
| **Floating-Point ALU (FPALU)** | 4 | FloatAdd – 2 cycles<br>FloatCmp – 2 cycles<br>FloatCvt – 2 cycles |
| **FP Multiply/Divide (FPMultDiv)** | 2 | FloatMult – 4 cycles<br>FloatMultAcc – 5 cycles<br>FloatMisc – 3 cycles |
| **Read/Write Port (RdWrPort)** | 4 | MemRead, MemWrite<br>FloatMemRead, FloatMemWrite<br>SIMD load/store operations |

Table 1: Functional Units and Operation Latencies

# 4 Implementation Details

Improvements in execution time can be achieved for programs where a portion of the integer arithmetic can be strategically offloaded to the floating-point pipelines, allowing FPUs and integer units to operate in parallel on the RISC-V Out-Of-Order processor. The following programs are used to demonstrate the main idea of speeding up programs using FP operations.

## 4.1 Matrix multiplication

### 4.1.1 Problem statement

Given two square matrices $A, B \in \mathbb{Z}^{n \times n}$, the objective is to compute their product $C = AB$, where $C \in \mathbb{Z}^{n \times n}$. Since both matrices are of identical dimensions, the matrix multiplication is well-defined.

### 4.1.2 Integer Version of Matrix Multiplication

A function named multiplyInt is implemented which takes the 2 matrices $A_{n\times n}$ and $B_{n\times n}$ (as 2 2-D arrays) as input and returns a matrix $C_{n\times n}$ (as a 2-D array) which is the result of matrix multiplication of A and B. The $i$-$j$th entry of the matrix $C$, denoted by $C_{ij}$, can be computed as

$$C_{ij} = \sum_{k=1}^{n} A_{ik}\, B_{kj}, \quad 1 \le i \le n,\ 1 \le j \le n$$

Integer Matrix multiplication is done using three nested for loops. The outer two loops, with indices $i$ and $j$ running over rows and columns of the matrix $C$ respectively, are required to compute each $C_{ij}$ of the matrix. For each $i$-$j$th entry, the innermost loop with index k computes the sum of products of the corresponding entries from row $i$ of the first matrix and column $j$ of the second matrix, and places the sum in the output 2-D array at the $i$-$j$th position.

Code Snippet

```
void multiplyInt(){
    for(int i = 0; i<n; i++){
        for(int j = 0; j<n; j++){
            int sum1 = 0;
            for(int k = 0; k<n; k++){
                sum1 += (matrixA[i][k]*matrixB[k][j]);
            }
            resultantMatrix[i][j] = sum1;
        }
    }
}
```

### 4.1.3 Float Versions

- **Version 1 (CompleteFloat):** In this version, multiplication and addition operations in the innermost loop are performed using FP operations. This version increases the execution time. The main reason for this is that, performing all multiplications and addition operations in the innermost loop using FP operations incurs more stalls when compared to that of integer version because of higher latency of FP operation(FloatMulAcc) than integer operation(IntMul+IntAdd). Conversions further slow down execution. As a result, this version fails to improve execution time.

  Code Snippet

  ```
  void multiplyIntegrated(){
      for(int i=0; i<n; i++){
          for(int j=0; j<n; j++){
             float sum1 = 0.0f;
             for(int k=0; k<n; k++){
              sum1 += ((float)matrixA[i][k]*(float)matrixB[k][j]);
             }
             resultantMatrix[i][j] = (int)sum1;
          }
      }
  }
  ```

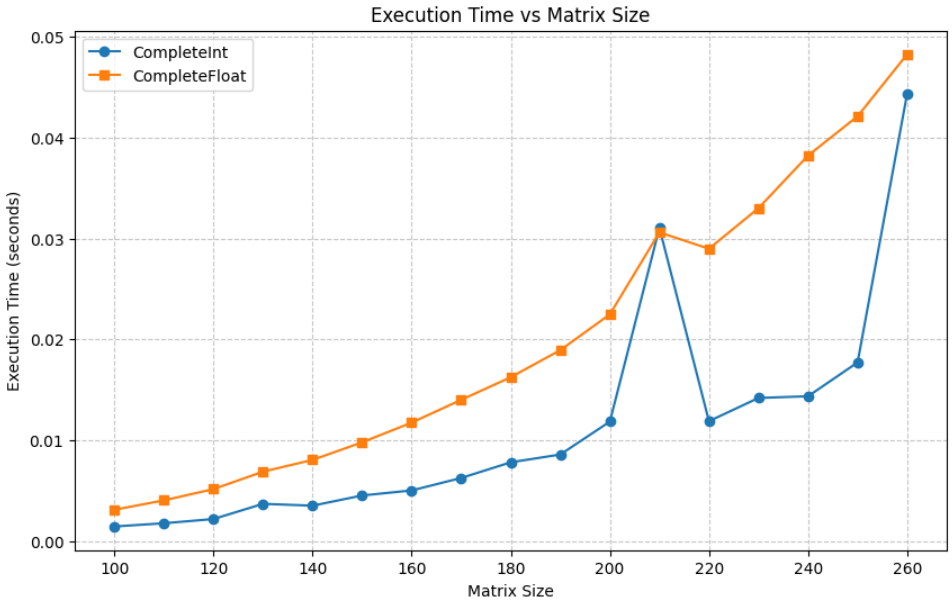



Figure 1: Execution Time vs Matrix Size for CompleteInt and CompleteFloat

- **Version 2 (Loop Unrolling):** In this version, the innermost loop is unrolled by 4 and some of the integer operations are replaced by FP operations. Although loop unrolling enables parallel execution of independent instructions across FP and integer pipelines, this version does not yield any significant improvement in execution time for the following reasons.

  1. The compiler's -O3 optimization performs loop unrolling in a better way (typically by a factor more than 4) in the **completeInt** program. However, in the programs where the innermost loop is manually unrolled, the compiler is restricted to the unrolling factor of 4. As a result, restricted unrolling prevents further optimization and gives no significant improvements.
  2. Offloading a portion of work to FP units(1Float in 4 or 2Float in 4) actually degraded the performance because the high FP latencies and conversion overheads outbalance the parallel execution gains. There is a further increase in execution time from 1Float offloading to 2Float offloading.




Figure 2: Execution Time vs Matrix Size for programs with Unrolling by factor 4

- **Version 3 (Swapping and Unrolling)**: In this version, three main changes are made in the integer version in order to achieve improvement in execution time.

  1. **Swapping:** In the code for matrix multiplication, the innermost two loops are swapped. Since the matrices are arranged in row-major order in C, the access of entries of a column of a matrix leads to cache misses. On swapping the innermost two loops of the standard matrix multiplication code, there is temporal locality of matrixA[i][k] and, spatial locality of matrixB[i][j] and resultantMatrix[i][j] for innermost loop iterating over j. This change reduces the execution time of the matrix multiplication, making the improvement in execution time obtained using FP operations more significant.
  2. **Unrolling:** The innermost loop over the induction variable j is unrolled by a factor of 10, which is determined to be the optimal value based on experiments with multiple unroll factors. Larger unroll factors may introduce issues such as instruction cache misses and increased register pressure due to register renaming.
  3. **Offloading Work:** Critical section (operations that affect the execution time significantly) in matrix multiplication code is multiplication and addition in the innermost loop.

     Critical Section

     ```
     resultantMatrix[i][j] += (matrixA[i][k]*matrixB[k][j]);
     ```

     These operations take **4 cycles**(IntMul-3 + IntAdd-1) in integer units. If these operations were to be performed using FP units, it requires **11 cycles** (3 floatcvt-2 each + floatMulAcc-5). The ratio of number of IntMulDiv units to number of FP-MultDiv units is **1:1**. Therefore, the ratio in which the work needs to be partitioned between FP units and integer units is $(4 : 11) : (1 : 1) =$ **(4:11)**. Since the unrolling factor is 10, **3** out of 10 unrolled operations are performed in FP pipelines and the remaining **7** operations are performed in integer pipelines. This leads to parallel execution of some operations in FP pipelines, resulting in improvement in execution time.

     **Why is the integer version with inner loops swapped and the innermost loop unrolled by 10 slower?**
     The execution time of this program is lower than the version where all 10 unrolled operations are performed using integer units because the number of IntMulDiv pipelines is less than the unroll factor(2 IntMulDiv pipelines).

Code Snippet for Integer Version

```
void multiplyInt(){
    for(int i=0; i<n; i++){
       for(int k=0; k<n; k++){
            int val = matrixA[i][k];
            int j = 0;
            for(;j<n-9;j+=10){
                resultantMatrix[i][j] += (val*matrixB[k][j]);
                resultantMatrix[i][j+1] += (val*matrixB[k][j+1]);
                resultantMatrix[i][j+2] += (val*matrixB[k][j+2]);
                resultantMatrix[i][j+3] += (val*matrixB[k][j+3]);
                resultantMatrix[i][j+4] += (val*matrixB[k][j+4]);
                resultantMatrix[i][j+5] += (val*matrixB[k][j+5]);
                resultantMatrix[i][j+6] += (val*matrixB[k][j+6]);
                resultantMatrix[i][j+7] += (val*matrixB[k][j+7]);
                resultantMatrix[i][j+8] += (val*matrixB[k][j+8]);
                resultantMatrix[i][j+9] += (val*matrixB[k][j+9]);
            }
            for(;j<n;j++){
                resultantMatrix[i][j] += (val*matrixB[k][j]);
            }
       }
    }
}
```

Code Snippet for Float Integrated Version

```
void multiplyIntegrated(){
    for(int i=0; i<n; i++){
       for(int k=0; k<n; k++){
            int val = matrixA[i][k];
            float floatVal = val;
            int j = 0;
            for(;j<n-9;j+=10){
                resultantMatrix[i][j] += (floatVal*matrixB[k][j]);
                resultantMatrix[i][j+1] += (floatVal*matrixB[k][j+1]);
                resultantMatrix[i][j+2] += (floatVal*matrixB[k][j+2]);
                resultantMatrix[i][j+3] += (val*matrixB[k][j+3]);
                resultantMatrix[i][j+4] += (val*matrixB[k][j+4]);
                resultantMatrix[i][j+5] += (val*matrixB[k][j+5]);
                resultantMatrix[i][j+6] += (val*matrixB[k][j+6]);
                resultantMatrix[i][j+7] += (val*matrixB[k][j+7]);
                resultantMatrix[i][j+8] += (val*matrixB[k][j+8]);
                resultantMatrix[i][j+9] += (val*matrixB[k][j+9]);
            }
            for(;j<n;j++){
                resultantMatrix[i][j] += (val*matrixB[k][j]);
            }
       }
    }
}
```

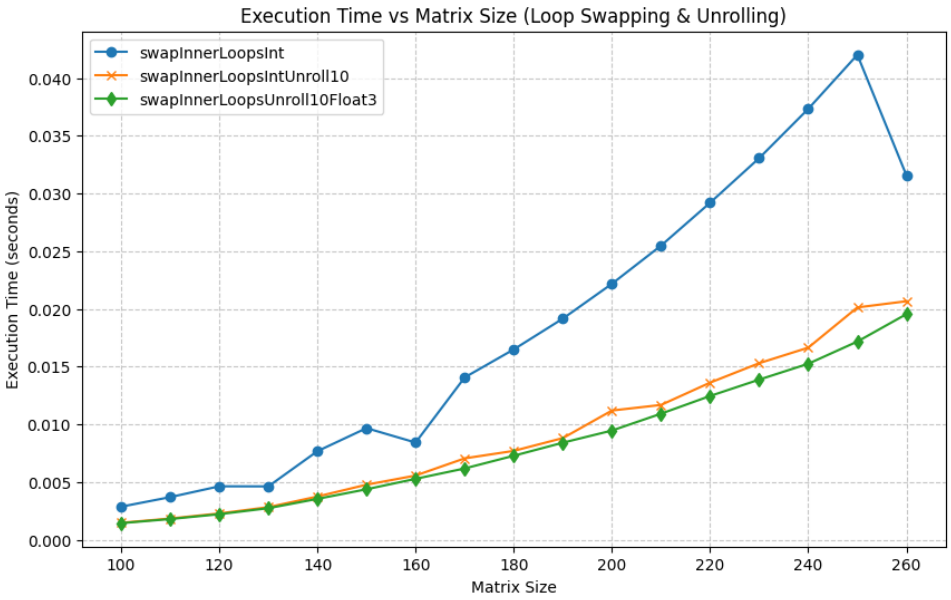



Figure 3: Execution Time vs Matrix Size for programs with swapped Inner loops

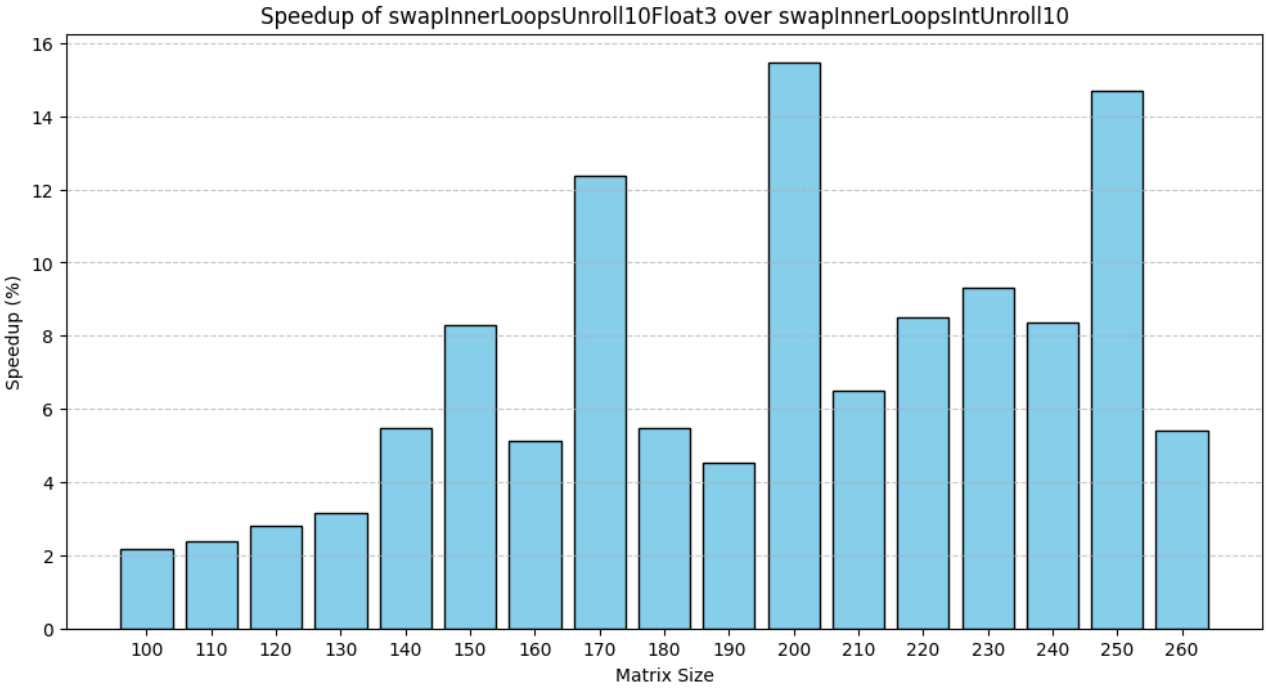



Figure 4: Speedup vs Matrix Size

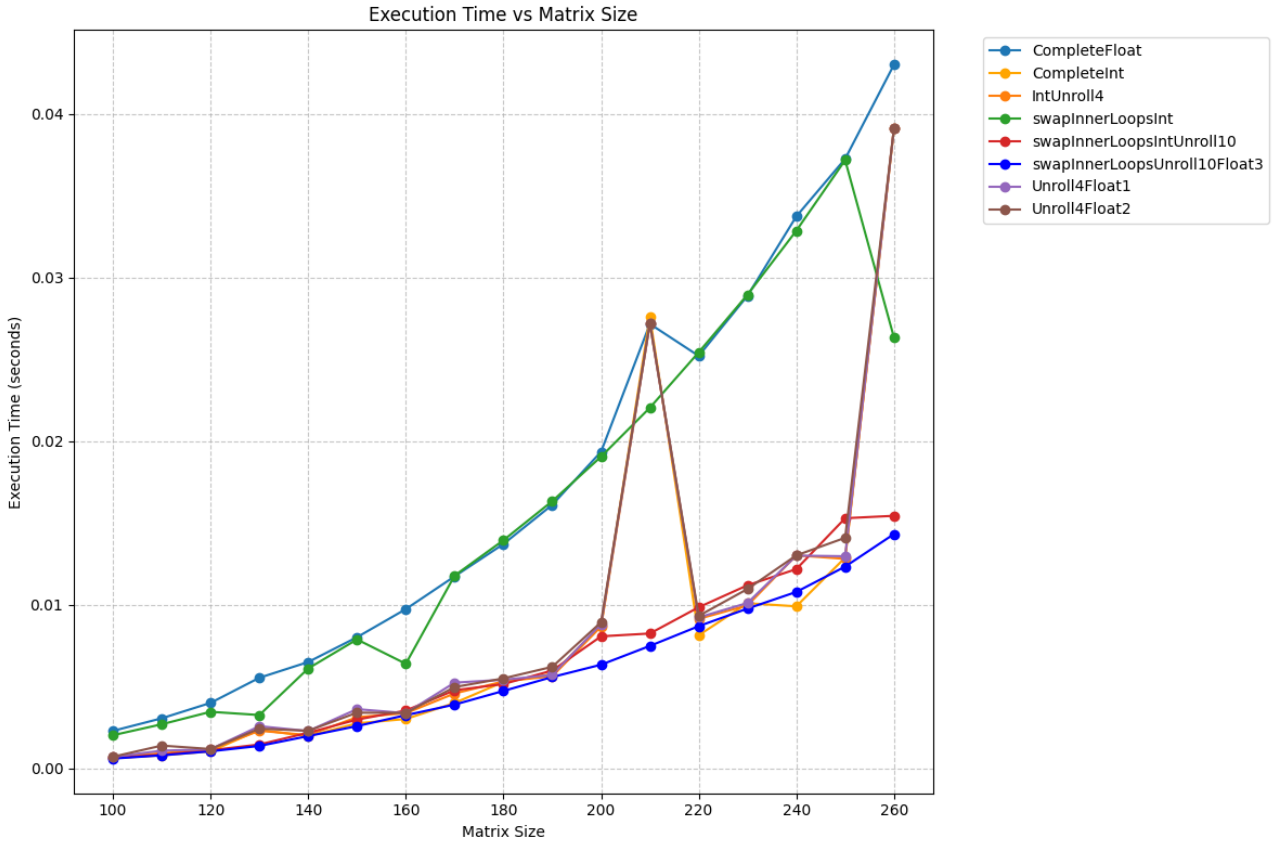



Figure 5: Execution Time vs Matrix Size for all versions

## 4.2 Minimum Element in an Array

### 4.2.1 Problem Statement:

Given an array of integers, the objective is to find the minimum element in the given array.

### 4.2.2 Integer Version

The algorithm iterates over the array, updating the current minimum whenever a smaller value is encountered, and outputs the final minimum at the end, which is the minimum element in the array.

### 4.2.3 Float Version

Critical section (operations that affect the execution time significantly) in the code for finding minimum element is the comparison operation.

Critical Section

```
for(int i = 0; i<n; i++){
    int value = arr[i];
    if(value < min1){
        min1 = value;
    }
}
```

The comparison operation takes **1** cycle in integer units and **4** cycles (floatCvt-2 + floatCmp-2) in FP units. The ratio of number of IntALU units to number of FPALU units is $(6 : 4) = (3 : 2)$. Because the IntALU units handle the initialization and update of induction variables, the former ratio can be considered **(1:1)**. Therefore, the ratio in which the work needs to be partitioned between FP units and integer units is **(1 : 4)**. As a result, the loop is unrolled by factor of 5 with 1 operation being performed in FP pipelines and 4 operations being performed in integer pipelines. This leads to parallel execution of some operations in FP pipelines, resulting in improvement in execution time.

**Why is the integer version with loop unrolled by 5 slower than normal Integer version?**

- Instruction Cache misses due to loop unrolling
- Spilling of registers due to register renaming
- Manual unrolling prevents the compiler from applying some optimizations.

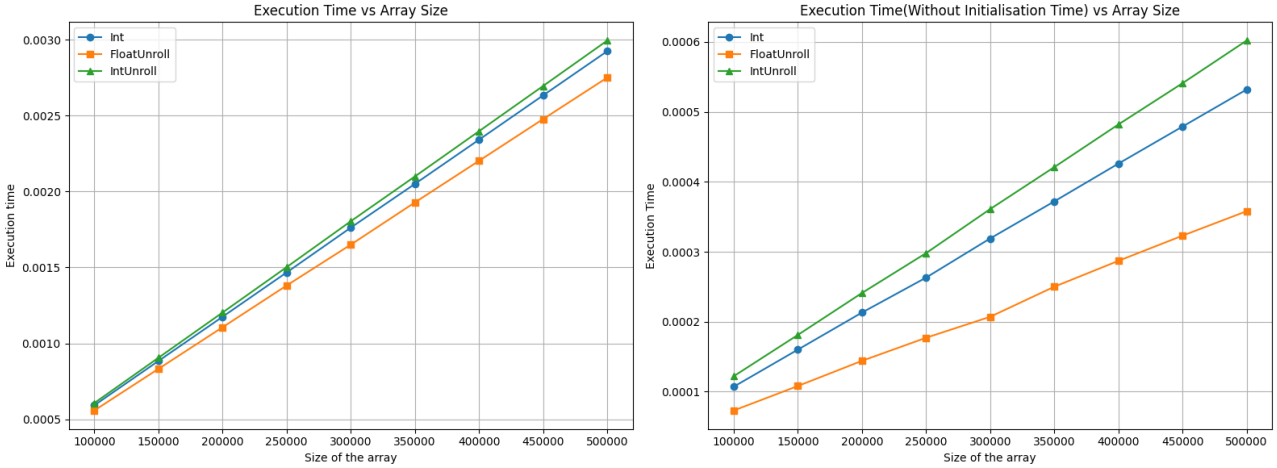



Figure 6: Execution Time vs Array Size

**Note:** The algorithm for finding minimum element in an array has same complexity as initialization of the array. Therefore, initialization time is subtracted from total execution time of the programs for correct performance analysis.

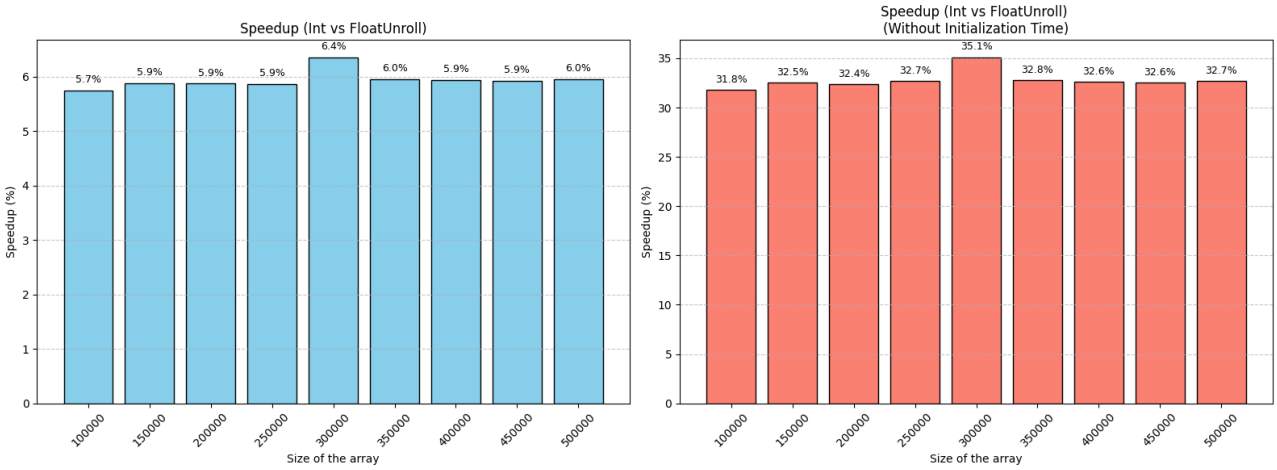



Figure 7: Speedup vs Array Size

## 4.3 1D integer convolution

### 4.3.1 Problem statement

We have to perform an integer convolution on a 2 1D arrays A and B.

### 4.3.2 Solution using normal C code

We do the integer convolution - summation of product of corresponding elements of the 1D array using 2 nested for loops iterating through each element of the matrices and generate the output array, which represents the 1D convolution of the two input arrays A, B.

### 4.3.3 Approach using our floating point pipelining optimization

Instead of doing a multiplication using multiple nested for loops directly, we split the innermost loop into many parts involving a summation of products. We will split these parts based on the value an integer k which directly depends on the ratio of number of FPUs to that of integer units and the ratio of their cycle counts in the processor being used.

$$k = \frac{\text{FPU}}{\text{IU}} \left( \frac{\text{IU}_{\text{cycles}}}{\text{FPU}_{\text{cycles}}} \right)^3$$

### 4.3.4 Speedup values

When we ran it for a random 1000 cross 1000 matrix, we noticed large variations in speedup values, below are some of the best case speed up times we have observed.
Under-O0 optimization: Average elapsed time for int: 0.003223200 seconds Average elapsed time for float: 0.002940400 seconds We got a speedup of around $(0.003223200 - 0.002940400)/0.003223200 * 100 = 8.8\%$
Under-O1 optimization, we noticed that for most cases we did not get any speedup.
Under-O2 optimization, we noticed that for most cases we did not get any speedup.
Under-O3 optimization Average elapsed time for int: 0.000324400 seconds Average elapsed time for float: 0.000271800 seconds We got a speedup of around $(0.000324400 - 0.000271800)/0.000324400 * 100 = 16.21\%$

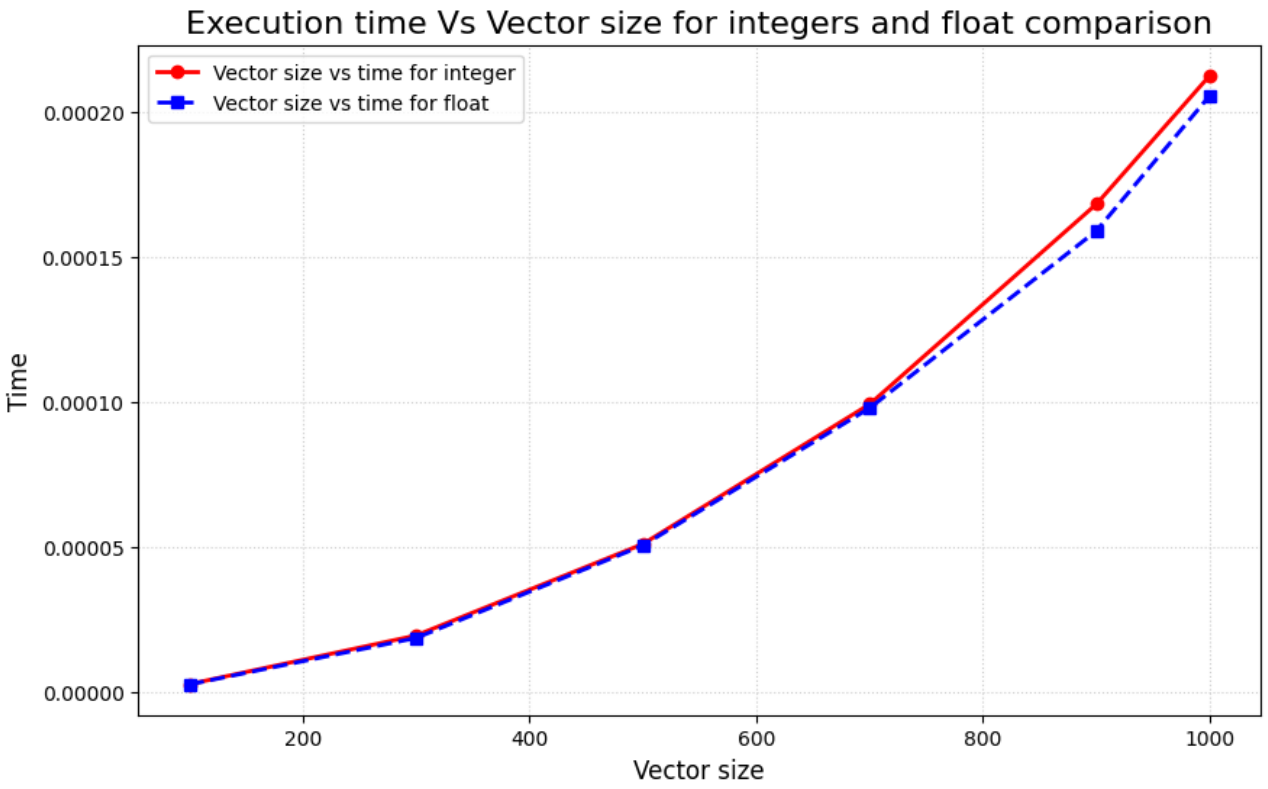
Following are the snippets of the programs

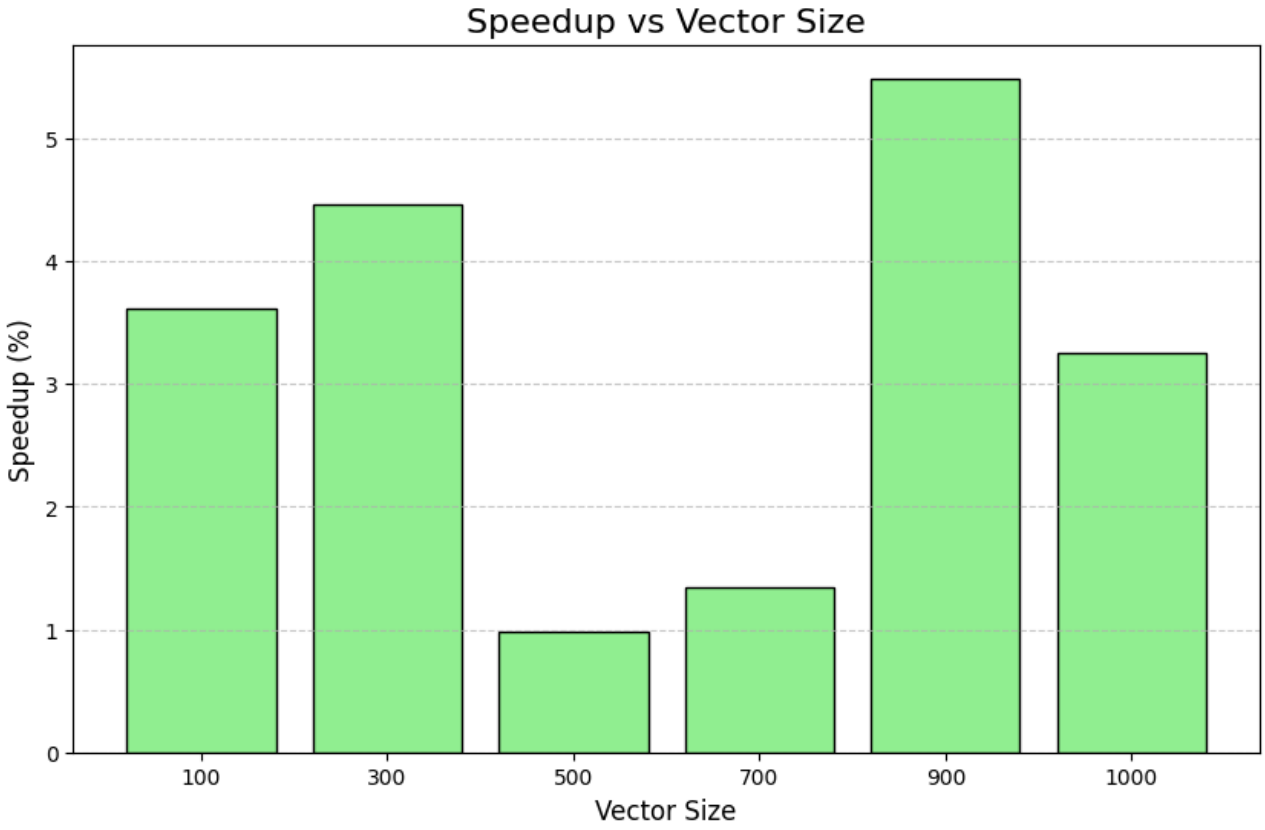
### Code Snippet for Integer Version

```
void convolution_int(){
    for(int i = 0; i<m; i++){
        for(int j = 0; j<n; j++){
            C[i+j] += (A[i]*B[j]);
        }
    }
}
```

### Code Snippet for Float Version

```
void convolution_float(){
    float k = pow((float)FPU/(float)IU, 1)*pow((float)IU_cycles/(float)FPU_cycles,3);
    float index = m*k/(k+1);
    int split_index = (int)index;
    for(int i = 0; i<m; i++){
        for(int j = 0; j<n; j++){
            if(i<split_index){
                float prod = (float)A[i]*(float)B[j];
                C[i+j] = C[i+j] + (int)prod;
            }
            else{
                C[i+j] = C[i+j] + (A[i]*B[j]);
            }
        }
    }
}
```






## 4.4 Factorial program

### 4.4.1 Problem statement

For a given integer n, we have to find it's factorial.

### 4.4.2 Floating point based approach

In the floating point-integer mixed approach, we split the for loop into 2 parts: one part looping from n to index + 1 (calculated by the integer unit) and the other part looping from index to 1 (calculated by the floating point unit). Note that the variable index = floor(n*($\frac{k}{k+1}$)) (between 1 to n) depends on the value of k, which is proportional to the ratio of the number of integer units to the number of FPU's and the ratio of the number of cycles taken by the integer unit to the number of cycles taken by the FPU for multiplication. Since, we just got that it is proportional and we do not know about the values of the powers, I decided to experiment with a few powers of the two ratios and see which one gives maximum speedup.

### 4.4.3 Observation

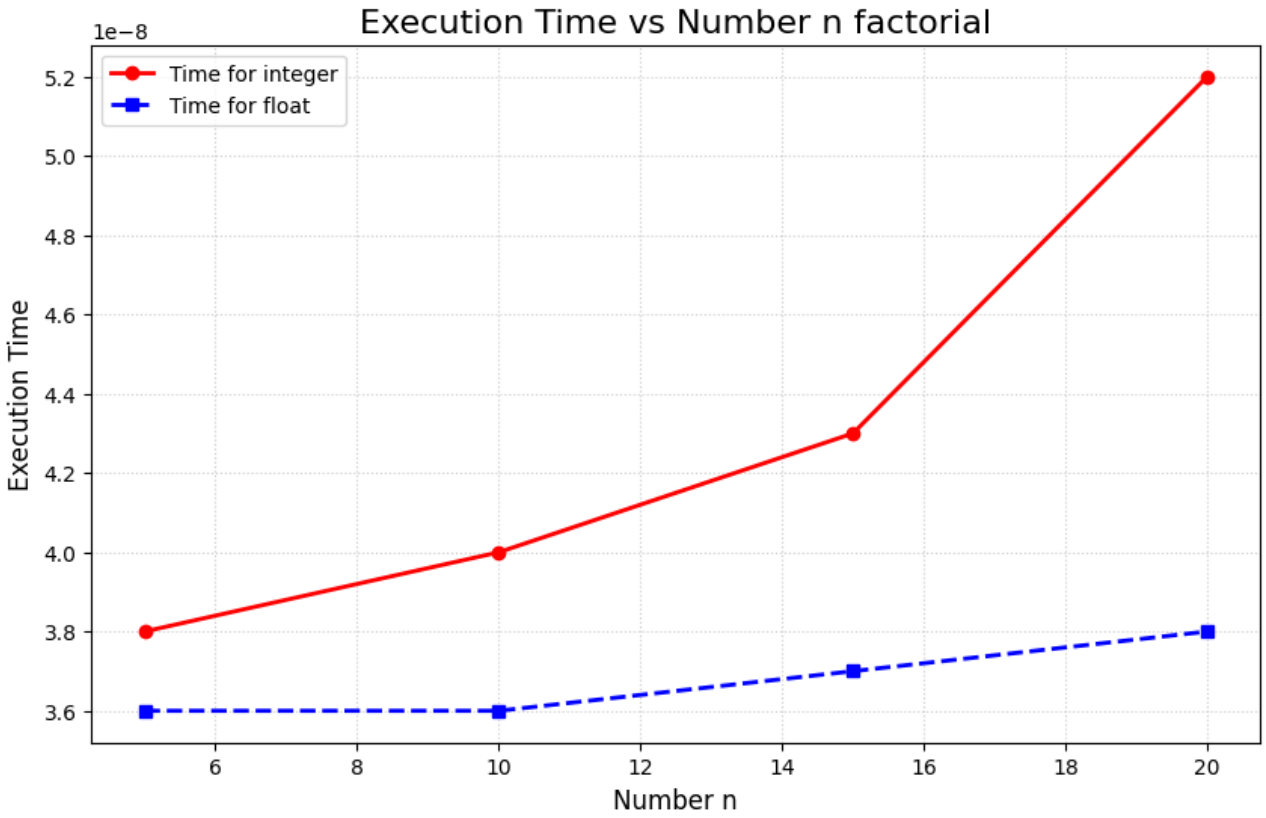
Initially, we observed no significant speedup or rather no speedup at all when we ran it using -O0 or -O1 optimization. However, as we changed the values of the powers in the proportionality equation, we noticed that at certain powers for a certain set of values of number of integer units, number of floating point units, number of cycles of integer units, number of cycles of floating point units, the perform of the mixed floating point + integer code increases relative to the integer only code. We also ran it using the -O2 optimization, where we noticed some

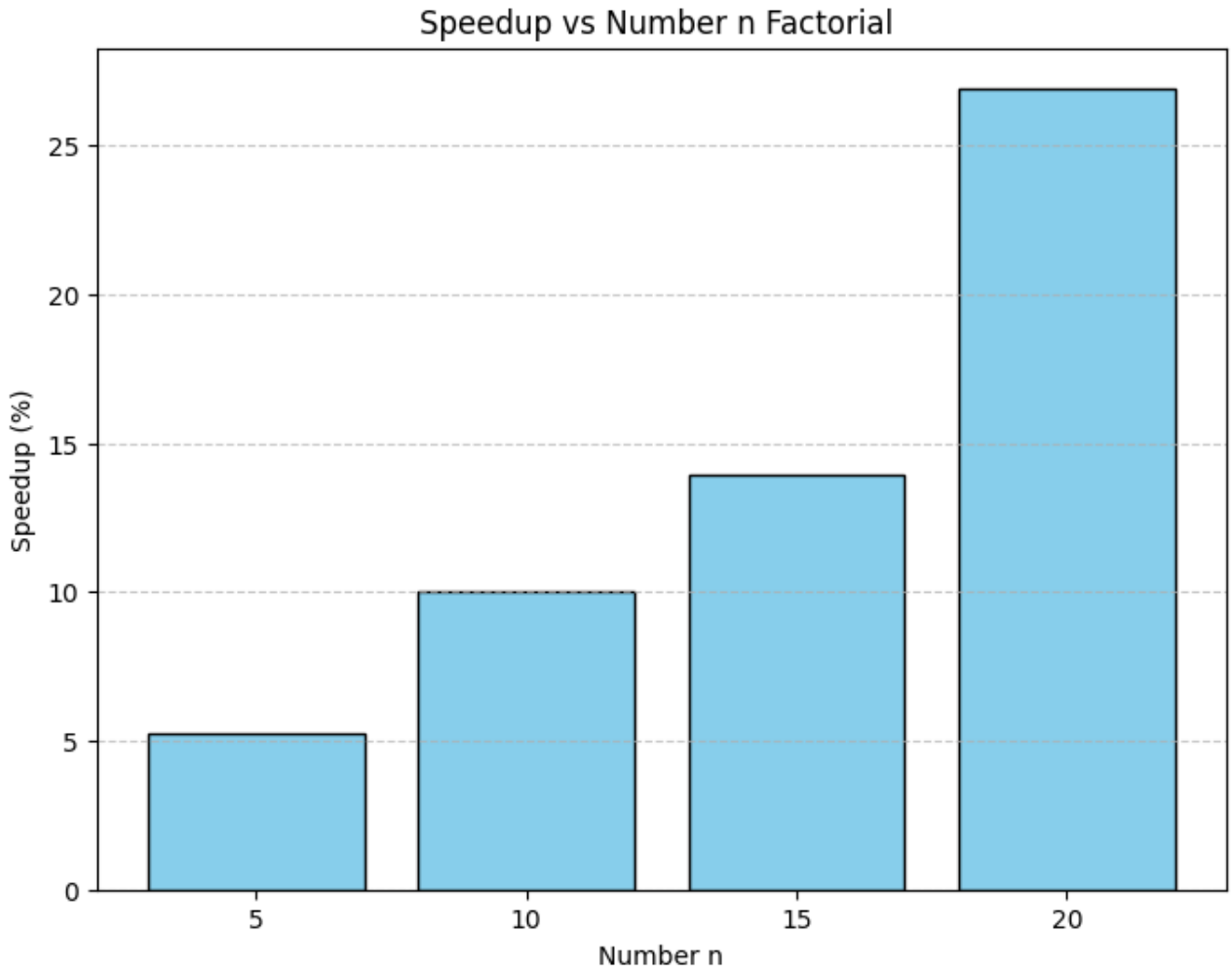
speedup. This speedup too increased with change in the values of the powers of the two ratios as parameters.
For example, we got Average elapsed time for fact_int(): 0.000000116 seconds Average elapsed time for fact_float(): 0.000000104 seconds giving a percentage decrease in time of around

$$\frac{(0.000000116 - 0.000000104)}{0.000000116} \times 100\% = 10.3\%.$$

However, while using the -O3 optimization, we noticed a significant speedup of around 25-30 percent. For example, we got Average elapsed time for fact_int(): 0.000000057 seconds Average elapsed time for fact_float(): 0.000000039 seconds giving a percentage decrease in time of around

$$\frac{(0.000000057 - 0.000000039)}{0.000000057} \times 100\% = 31.57\%.$$






# 5 Limitations

- The floating-point versions of these programs are valid only when the computed results lie within the range $[-(2^{24}-1),\ 2^{24}-1]$ because of the limited precision of the 23-bit mantissa in single-precision floating-point arithmetic. Beyond this interval, the representation cannot guarantee correctness leading to accuracy loss.
- Some float version programs show improvement in execution time only over a particular range of inputs.
- Speedup is observed only when the code is compiled with the O3 optimization level.

# 6 Work Partition

## Nehith

- **Programs:** Matrix Multiplication and finding minimum element in an array, and their corresponding implementation details in the report

## Krishnan

- **Programs:** Integer Convolution(1D) and Factorial, and their corresponding implementation details in the report